

FORGE / ALGORITHMIC EXTENSION

Forge Beyond Word Closure

Certified multilinear summaries for recursive proofs,
context-free traces, and observable quotients

A concrete design, executable prototypes, and reproducible evidence

September 15, 2026

Abstract

Forge already has substantial proposals for structural proof search, algebraic certificates, word-observation closure, and finite-algebra covers. This article adds a specific algorithmic layer rather than another general architecture: an exact, many-sorted *multilinear reachable-span* worker. It computes finite certificates for recursive definitions with several independently recursive children, even when their rational-valued summaries take infinitely many values. A matrix-valued grammar compiler extends the same worker to context-free restrictions on execution traces. A second closure, generated by one-hole contexts, constructs exact observable quotients and verifies them through finite commuting diagrams.

The package implements these algorithms in standard-library Python, a multi-affine source frontend, an independently organized checker, and compressed derivation witnesses. Recorded evidence includes 240 exhaustive finite-language differential cases, 40 quotient certificates, 20 deliberately invalid objects rejected, and eight replayed fixtures. A 61-node witness represents a word of length 2^{60} ; a noisy five-coordinate tree summary reduces to two observable coordinates. The mathematical foundation is established multiplicity-tree-automata theory, adapted here to Forge's certificate interface. No new general automata theorem, implemented Lean tactic, Lean-kernel validation, or performance comparison with `grind` is claimed. One hand-written Lean integration specimen is included and explicitly uncompiled.

Repository baseline. Forge commit

c98e47c5f804e92880fc1d0e378c1b95832b685c.

Delivered implementation. Exact search, source frontends, independent Python replay, tests, recorded certificates, and article sources.

Contents

1	The contribution relative to the inspected repository	1
1.1	What this proposal deliberately does not repeat	1
1.2	Established mathematics, new engineering application	2
1.3	Relationship to <code>grind</code>	2
2	Two goals that expose the gap	2
2.1	Exact tree counts without a finite carrier	2
2.2	Balanced traces are not unrestricted words	3
3	The many-sorted multilinear fragment	3
3.1	Signatures, derivations, and observations	3
3.2	Reachable spans, not arbitrary reachable sets	4
4	A semi-naive exact closure algorithm	5
4.1	State and work scheduling	5
4.2	Termination and completeness	5
4.3	Height and compressed witness size	6
4.4	The cost model must not hide integer growth	7
5	The finite acceptance language	7
5.1	Positive certificates contain identities, not a search transcript	7
5.2	Coverage is a mathematical obligation	8
5.3	Counterexamples are original derivations	8
5.4	An independent source audit	8
5.5	Decoder discipline and resource limits	9
6	A context-free source compiler	9
6.1	Matrix semantics of a production	9
6.2	Observations and comparisons	10
6.3	Orientation errors need a noncommutative control	10
6.4	Exponential terminal length with a small exact witness	10
7	The multi-affine recursive-source frontend	11
7.1	A mechanically checkable source fragment	11
7.2	What is rejected, and why	12
7.3	A practical recognition algorithm for Lean	12
8	Exact observable quotients by one-hole contexts	12

8.1	Why forward reachability is only the first compression	12
8.2	Work in reachable coordinates	13
8.3	The context kernel is a congruence	13
8.4	Constructing a concrete small model	14
8.5	What the quotient certificate actually checks	14
8.6	A recorded compression with visible semantics	15
9	A concrete Lean integration plan	15
9.1	The missing theorem is finite span closure	15
9.2	Proposed module boundaries and acceptance gates	15
9.3	A staged proof-construction strategy	16
9.4	The source-correspondence chain	17
9.5	Library choices	17
10	Leant and Djex: a narrow, useful connection	18
11	Executed experiments and what they establish	18
11.1	Environment and reproducibility	18
11.2	Recorded fixtures	19
11.3	An exhaustive differential oracle	19
11.4	A separate paired-derivation quotient oracle	20
11.5	Negative controls and parser tests	20
11.6	Isolation and the strength of the evidence	20
12	Integration priorities, limits, and further development	21
12.1	A minimal merge that would be worth shipping	21
12.2	Where the practical limits arise	21
12.3	Two next algorithmic improvements with explicit contracts	22
13	Reproducing and extending the package	22
14	Conclusion	23
A	Certificate schema and implementation map	23
B	Snapshot and evidence notes	24

1 The contribution relative to the inspected repository

1.1 What this proposal deliberately does not repeat

The inspected Forge baseline is already a merger of eighteen proposals, not a small sketch. Its existing closure section handles unrestricted words through finite-dimensional observable spaces, nonlinear transitions through polynomial ideals, and inductive data through finite-algebra covers. Its status ledger also records continuation-local synthesis, cyclic descent, integer projection, finite Kripke models, and recurrence transport. Those are prerequisites here, not contributions of this article [1, 2, 3].

A particularly important distinction comes from the existing file `lean/Forge/Closure/Covers.lean`. Its theorem `ForgeClosure.tree_property` already lifts a closed predicate on an arbitrary carrier to a property of every binary tree. There is no need to invent another version of that induction principle. What is missing from the inspected cover design is the concrete construction below: a finite generating family for an *infinite* rational carrier, with constructor closure checked only on basis tuples. Multilinearity makes those finitely many checks sufficient [4].

Existing ingredient	Additional mechanism	What changes for the user
Unrestricted-word observable closure	Many-sorted, several-child multilinear closure	Decides zero observations of supported tree folds, not only unary transition sequences.
Finite-carrier algebra cover	Span cover of an infinite rational carrier	Can prove exact unbounded counting identities without enumerating carrier values or reducing modulo an integer.
Regular-language product restriction	Context-free matrix grammar	Keeps balanced calls, nested syntax, and mutually recursive trace constraints rather than dropping them.
Unary backward observations	Sibling-conditioned one-hole contexts	Removes internal summary coordinates invisible to every surrounding supported computation.
Source compatibility obligations	Executed multi-affine and CFG source frontends with independent audits	Supplies two concrete, narrowly specified reification targets for a future Lean adapter.
Existing cover induction theorem	Finite coefficient replay and span-closure lemma	Reuses the already available logical lifting, instead of replacing it.

The review was anchored to the merged README, implementation ledger, structural and closure material, the Leant/Djex comparison, and the existing cover code. It was not a line-by-line audit of every frozen submission. Repository code search returned an explicitly incomplete result, so it is not used as evidence of exhaustive absence. The novelty claim is the *specific integrated worker, certificate formats, source audits, and executed evidence* relative to those inspected capabilities, not a claim that no related phrase occurs anywhere in the archive.

1.2 Established mathematics, new engineering application

Multiplicity tree automata provide the relevant foundation: constructor semantics are multilinear, reachable values span a finite-dimensional space, and distinguishing trees may admit small DAG representations despite exponential expanded size. The distinction between unit-cost field operations and ordinary bit complexity is essential; the equivalence problem has a close connection to polynomial identity testing [6]. This article gives self-contained proofs of the particular many-sorted certificate obligations used here, rather than claiming discovery of the underlying automata algorithms.

The engineering delta matters independently of mathematical priority. A proof assistant needs a precise source fragment, an authoritative input independent of the certificate, a finite acceptance language, a path back to the original goal, and tests that challenge each of these boundaries. The following sections supply those details, together with code that actually runs.

1.3 Relationship to grind

The official description of `grind` emphasizes cooperating reasoning procedures on a shared local state [7]. The proposed worker is not another local simplifier. It changes an infinite structural obligation into a finite table of rational identities and then returns those identities to the proof layer. Local arithmetic remains useful for checking the entries. The intended capability gain is therefore a change of proof representation, not an unsupported assertion that the prototype is faster than `grind`.

2 Two goals that expose the gap

2.1 Exact tree counts without a finite carrier

For a full binary tree t , let $L(t)$ count leaves and $I(t)$ count internal nodes. Their defining equations are

$$(L, I)(\text{leaf}) = (1, 0), \quad (L, I)(\text{node}(u, v)) = (L(u) + L(v), I(u) + I(v) + 1).$$

The desired property is $L(t) - I(t) - 1 = 0$ for every finite tree. Neither component has a finite range. A finite-state abstraction modulo m can establish a congruence, but that is not the requested rational or integer equality.

Introduce a homogeneous coordinate and write a summary as (c, ℓ, i) . The leaf summary is $(1, 1, 0)$ and the binary constructor is

$$F((c_1, \ell_1, i_1), (c_2, \ell_2, i_2)) = (c_1 c_2, \ell_1 c_2 + c_1 \ell_2, i_1 c_2 + c_1 i_2 + c_1 c_2). \quad (1)$$

This map is bilinear in its two *child slots*. Actual tree summaries have $c = 1$, but arbitrary linear combinations need not. That is precisely why the homogeneous coordinate must remain in the constructor formula.

The prototype discovers the two reachable generators

$$b_0 = (1, 1, 0), \quad b_1 = (1, 2, 1),$$

and the complete constructor table

input	output in (b_0, b_1) coordinates
leaf	$(1, 0)$
(b_0, b_0)	$(0, 1)$
(b_0, b_1)	$(-1, 2)$
(b_1, b_0)	$(-1, 2)$
(b_1, b_1)	$(-2, 3)$.

(2)

Both generators are annihilated by $q = (-1, 1, -1)$. These five identities, bilinearity, and the existing tree-cover induction principle settle every tree. Negative coefficients are harmless: this is linear span, not a convex hull or a nonnegative cone.

2.2 Balanced traces are not unrestricted words

Consider column-state updates

$$A = \begin{pmatrix} 1 & 1 \\ 0 & 1 \end{pmatrix}, \quad B = \begin{pmatrix} 1 & -1 \\ 0 & 1 \end{pmatrix},$$

with a denoting A and b denoting B . The grammar

$$S \longrightarrow \varepsilon \mid aSb \mid SS \tag{3}$$

generates balanced-parenthesis words. Every such word has identity transfer matrix, so its upper-right entry vanishes. The unrestricted word a does not have that property.

Dropping the grammar restriction would therefore turn a true theorem into a false stronger statement. A bounded-depth grammar unrolling is also inadequate for the unbounded theorem. The worker instead maintains a subspace of 2×2 matrices for each nonterminal. The three constructor maps are

$$\varepsilon \mapsto I, \quad X \mapsto BXA, \quad (X, Y) \mapsto YX.$$

They are respectively nullary, linear, and bilinear. In this example the reachable matrix space has dimension one, and three closure identities suffice. With the return matrix changed to I , the same worker returns the legal word ab with upper-right entry 1.

These examples are deliberately small enough to inspect by hand. Their significance is not that a mathematician needs automation to prove them; it is that they identify a stable computational fragment and furnish exact acceptance and rejection fixtures for its implementation.

3 The many-sorted multilinear fragment

3.1 Signatures, derivations, and observations

Let $\mathcal{S}$ be a finite set of sorts. Associate to sort s the vector space $V_s = \mathbb{Q}^{d_s}$, allowing $d_s = 0$. A finite signature consists of operations

$$p : (s_1, \dots, s_k) \rightarrow s, \quad F_p : V_{s_1} \times \dots \times V_{s_k} \rightarrow V_s,$$

where F_p is linear in each input separately. Nullary operations denote fixed vectors. The number k is the operation's arity. A derivation is a finite, well-sorted tree built from these operations; its value

is determined recursively by F_p . Repeated appearances of a sort in the signature denote *independent child occurrences*, not a constraint that the two children be equal.

A target is a pair (s, q) with $q : V_s \rightarrow \mathbb{Q}$ linear. There may be several targets at several sorts. The decision question is

$$\forall (s, q) \text{ listed, } \forall t : s, \quad q(\llbracket t \rrbracket) = 0. \quad (4)$$

An affine observation becomes linear after homogenization. Equality of two compatible fold computations becomes a zero observation of their paired summary. Operations on the paired carrier apply each implementation's constructor to its respective coordinate projections; these remain multilinear. No equality of their private state coordinates is required.

3.2 Reachable spans, not arbitrary reachable sets

Define

$$\mathcal{R}_s = \{\llbracket t \rrbracket : t : s\}, \quad W_s = \text{span}_{\mathbb{Q}} \mathcal{R}_s.$$

The set $\mathcal{R}_s$ can be infinite and need not be a vector subspace. Nevertheless, W_s is finite-dimensional. The exact abstraction works because multilinear operations preserve the linear consequences of independent child substitutions.

Lemma 3.1 (Basis-tuple extension). *Let B_{s_j} be finite lists of vectors generating subspaces U_{s_j} . If*

$$F_p(b_{1,i_1}, \dots, b_{k,i_k}) \in U_s$$

for every tuple of generators, then $F_p(U_{s_1}, \dots, U_{s_k}) \subseteq U_s$. For a nullary operation the hypothesis is simply that its value belongs to U_s .

Proof. Write $x_j = \sum_i \lambda_{j,i} b_{j,i}$. Repeatedly distribute in the input slots:

$$F_p(x_1, \dots, x_k) = \sum_{i_1, \dots, i_k} \left(\prod_{j=1}^k \lambda_{j,i_j} \right) F_p(b_{1,i_1}, \dots, b_{k,i_k}). \quad (5)$$

Every summand lies in U_s , which is a subspace. If an input space has no generators, it is the zero space; an operation with that input evaluates to zero, and the empty sum is correct. With no input slots there is one empty tuple, not zero tuples, so the constant obligation remains. $\square$

Proposition 3.2 (Least closed family). *The family (W_s) is the least family of subspaces containing all nullary values and closed under every F_p .*

Proof. A closed family contains the value of every derivation by induction, hence contains its span. Conversely, expand each input from W_{s_j} as a finite linear combination of actual derivation values. Every term in (5) is the value of the derivation obtained by applying p to the selected child derivations. Thus it lies in $\mathcal{R}_s$ and their linear combination lies in W_s . $\square$

The independence of child choices is doing real work in the converse. A source operation that insists on applying a constructor to the *same* subtree twice has a correlated domain; treating those occurrences as independent may be a sound over-approximation for proving a universal property, but it is not an exact decision reduction or an automatic source of real counterexamples.

4 A semi-naive exact closure algorithm

Implementation status. Implemented in `prototype/search.py`; exercised by the recorded suites.

4.1 State and work scheduling

For each sort maintain an incremental echelon structure, a list of original generating vectors, and a witness-node identifier for each generator. Keep the vectors admitted to the generator list *unchanged*. Echelon reduction rows are auxiliary arithmetic objects and need not themselves be reachable.

Each work item is $(p, i_1, \dots, i_k)$, referring to original generator indices for the operation's input sorts. Start by enqueueing the empty tuple of every nullary operation. When a new generator of sort s is admitted, revisit every occurrence of s in every operation signature. Pin that position to the new index and range the other positions over the current generator lists. A set of previously scheduled keys prevents duplicate work.

```

for each sort s: B[s] := empty; echelon[s] := empty
queue := every nullary operation with its empty child tuple
while queue is nonempty:
  check configured work limits
  (p, child_indices) := pop_front(queue)
  v := F[p](the referenced original generator vectors)
  check exact coefficient-size limit
  if a target at output_sort(p) has nonzero value on v:
    return an original-derivation DAG ending at this operation
  if v belongs to span(B[output_sort(p)]):
    continue
  append v and its derivation witness to that sort's generator list
  update the incremental echelon structure
  for every matching input occurrence in every operation:
    schedule previously unseen tuples using the new generator there
assemble closure coefficients for every final generator tuple
return all closure identities and all target-annihilation identities

```

The two recursive positions of a binary operation must both trigger scheduling. Treating a repeated sort as a single occurrence misses cross terms. The `slot_sensitive` test makes this error visible with a non-symmetric operation.

The implementation does a second exact pass over the final tuple table to produce coordinate witnesses. This does not change the search bound but should be included in production cost accounting. Search statistics record candidate evaluations separately from the final identity-assembly pass; they are not claimed to count all arithmetic operations.

4.2 Termination and completeness

Put $N = \sum_s d_s$ and let $r_s = \dim W_s$.

Theorem 4.1 (Exact coverage of the fragment). *With no operational cutoff, reachable-span closure terminates and decides (4). A positive result has a finite constructor-closure certificate. A negative result has a well-sorted finite derivation with a nonzero requested observation.*

Proof. Each admitted vector is linearly independent of previously admitted vectors at its sort, so at most N admissions occur. Only finitely many operations and final generator tuples exist. Each work key is processed once. Every admitted vector is a genuine derivation value: its children refer to earlier admitted witness nodes, and a nullary value is already a derivation. Thus the computed span is contained in W_s .

At termination, every final generator tuple has been scheduled. Indeed, the last of its constituent generators to be admitted scheduled that tuple in at least one input position. All results are therefore in the output span. Lemma 3.1 and Proposition 3.2 imply that the computed spans equal W_s .

If every target vanishes on every admitted generator, linearity makes it vanish on W_s and hence on every derivation. If the algorithm observes a nonzero value, the carried derivation is a counterexample. If the property were false but every candidate observation were zero, the completed generating family would contradict the preceding implication. Therefore one of these two outcomes occurs. $\square$

A budget cutoff is neither branch of this theorem. The executable returns UNKNOWN without a certificate in that case. In particular, a coefficient-size limit can stop a computation even when the number of basis admissions is tiny.

4.3 Height and compressed witness size

Let $W_s^{(h)}$ be the span of values of derivations of height at most h , with nullaries at height zero, and set $W_s^{(-1)} = 0$. If the family at two consecutive heights is equal, Lemma 3.1 makes it closed, so the chain has stabilized permanently.

Corollary 4.2 (A height bound, not a word-length bound). *If a target is false and $N \geq 1$, there is a counterexample derivation of height at most $N - 1$.*

Proof. Falsity requires some nonzero reachable value, and hence a nonzero nullary value somewhere: multilinearity maps a tuple containing zero to zero. Thus the chain from $W^{(-1)}$ to $W^{(0)}$ has at least one dimension increase. Until stabilization, each subsequent step increases the total dimension by at least one. Total dimension is at most N , so $W^{(N-1)}$ is the full reachable family. A nonzero linear functional on this span must be nonzero on at least one of its generating derivation values. $\square$

The algorithm's witness DAG has at most N nodes when it returns a counterexample. Before the final node, every node represents one independent admission. At the final sort, the nonzero observed vector cannot lie in the span of earlier generators, because that target vanished on each of them. The last node therefore fits within the same total dimension budget. Unused earlier witness nodes may be removed, but the prototype does not promise a smallest DAG or a shortest word.

An arity- a derivation tree of height h can have exponentially many nodes in h . Moreover, a grammar production can emit several terminals even with no recursive child. Consequently, a small ambient dimension or a linear height bound does not justify imposing the unrestricted-word bound $D - 1$ on *terminal length*.

4.4 The cost model must not hide integer growth

The number of distinct constructor tuples is

$$K = \sum_{p:(s_1, \dots, s_k) \rightarrow s} \prod_{j=1}^k r_{s_j}, \quad (6)$$

where an empty product is one. For explicitly stored sparse tensors, evaluating a tuple takes at most its tensor's number of entries times its arity, plus output accumulation. A straightforward echelon membership check at sort s takes $O(d_s r_s)$ field operations. Thus bounded arity, moderate dimension, and sparse constructors give a practical finite algorithm.

This is a *field-operation* discussion, not a deterministic polynomial bit-time claim. Even the acyclic scalar grammar

$$N_0 \rightarrow 2, \quad N_i \rightarrow \text{multiply}(N_{i-1}, N_{i-1})$$

produces 2^{2^i} , whose binary representation has $2^i + 1$ bits. A small derivation DAG does not compress its fully expanded rational value. The recorded 64-bit coefficient cutoff returns UNKNOWN on this family rather than quietly accepting an approximate answer. This distinction agrees with the complexity caveat in multiplicity-tree-automata theory [6].

5 The finite acceptance language

Implementation status. Independent decoder and replay implemented in `prototype/checker.py`. The checker imports neither the producer nor the source compiler.

5.1 Positive certificates contain identities, not a search transcript

For each sort let $B_s \in \mathbb{Q}^{d_s \times r_s}$ be a matrix whose columns are the supplied generators. For each operation and every tuple of input column indices, the certificate supplies $h_{p,i} \in \mathbb{Q}^{r_s}$ satisfying

$$F_p(B_{s_1} e_{i_1}, \dots, B_{s_k} e_{i_k}) = B_s h_{p,i}. \quad (7)$$

Every target additionally requires

$$qB_s = 0. \quad (8)$$

The checker obtains F_p , operation signatures, sorts, and targets from the separately supplied original problem. It does not read an operation definition or a replacement target from the certificate.

Theorem 5.1 (Positive certificate soundness). *If all identities (7) and (8) hold with complete operation/tuple coverage, then (4) holds.*

Proof. Take $U_s = \text{im } B_s$. Equation (7) and Lemma 3.1 show constructor closure, including all nullaries. Induction on a derivation shows $\llbracket t \rrbracket \in U_s$. Equation (8) makes q zero throughout U_s . $\square$

No rank calculation occurs in this proof. In particular, the positive checker must *not* require that the supplied columns be independent, minimal, reachable, or produced by the prescribed search. Any finite enclosing subspaces inside the target kernels are sufficient. The tests explicitly accept a redundant two-column cover of a one-dimensional carrier. Rank is a search optimization; it is not an admission requirement for this certificate kind.

5.2 Coverage is a mathematical obligation

The checker reconstructs the required set of keys

$$\{(p, i_1, \dots, i_k) : 0 \leq i_j < r_{s_j}\}$$

from the original signature and the supplied generator-list lengths. A closure table must contain exactly those keys, each once. Checking only the entries that a producer happens to supply would let it omit a difficult mixed tuple. For example, checking $F(b_0, b_0)$ and $F(b_1, b_1)$ does not establish bilinear closure on $F(b_0, b_1)$.

The empty cases deserve explicit treatment. If an operation has a zero-dimensional input span and positive arity, there are no generator tuples, and multilinearity forces its value on that input space to zero. A nullary operation has *one* empty tuple. Even a zero nullary vector therefore has a closure entry, whose coefficient list may itself be empty. An unproductive grammar with no finite derivations can validly have empty generator lists everywhere; that is different from a productive grammar whose every value is zero. Both cases occur in the tests.

5.3 Counterexamples are original derivations

A negative certificate contains a topologically ordered array of nodes. Each node names an original operation and earlier child-node indices of the required sorts. The checker evaluates that original operation, records the result and output sort, and eventually checks the selected root against the selected original target. The claimed observation must equal the recomputed rational and must be nonzero.

The list never contains a floating-point witness, a symbolic span combination masquerading as an input, or a producer-supplied truth table. For the CFG frontend the checker also computes terminal length by dynamic programming on the DAG. It does not expand the word. An expanded tree-size calculation similarly counts repeated child occurrences even when they share a stored node.

The accepted diagnostic means: *this derivation of the supplied abstract source has a nonzero observation*. A future native adapter must separately establish that the derivation denotes a legal original Lean input. With an exact source correspondence, the witness can support a proof of failure of the universally quantified source assertion. With only an over-approximation, it is merely a potential counterexample until source replay succeeds.

5.4 An independent source audit

A JSON problem may contain both a source description and a compiled tensor. The producer uses the tensor for efficient search. The checker instead interprets the source independently when checking constructor identities and witnesses. It also verifies agreement between the source and tensor on all *ambient standard-basis tuples*. Because both interpretations are explicitly multilinear, this finite audit establishes agreement on all inputs of the declared vector spaces.

The CFG checker executes each original rule as column-state actions, once for each initial coordinate vector. The compiler uses matrix multiplication to build the tensor. These are different organizations of the computation and share no implementation function. The multi-affine checker interprets the original monomials directly, including the homogeneous coordinates for omitted slots; it does not trust the compiler's collected tensor entries.

This protects the delivered source languages, not arbitrary Lean expressions. A correct interpretation of the wrong extracted source still proves the wrong theorem. The remaining Lean-to-source correspondence obligation is therefore explicit in Section 9.

5.5 Decoder discipline and resource limits

Rationals are canonical strings such as `"-3/2"` or `"0"`; floating-point values, zero denominators, noncanonical `"2/2"`, and nonfinite JSON constants are rejected. The JSON loader rejects duplicate object keys. Booleans are not accepted as integer indices even though Python treats `bool` as an integer subclass. Operation names are unique, tensor coordinates are bounded, explicit zero entries and duplicate tensor entries are rejected, and unknown object fields fail closed.

The current checker caps ambient dimension at 256 per sort, constructor arity at four, enumerated frontend and closure tuples at 200,000, rational input size at 16,384 bits, and JSON input at 16 MiB per file. These are declared implementation limits, not mathematical limits of the method. They do not constitute hostile-input hardening: exact arithmetic can allocate large intermediate integers before a later check, and aggregate object sizes still need tighter production controls. A service deployment should isolate the checker process and impose memory and execution limits independently of its mathematical checks.

6 A context-free source compiler

Implementation status. Implemented in `prototype/compilers.py`, with independent source replay and a tensor audit.

6.1 Matrix semantics of a production

Assign a rational $n \times n$ matrix M_a to each terminal a . Reading a word from left to right updates a column vector, so

$$M_{a_1 \dots a_m} = M_{a_m} \cdots M_{a_1}, \quad M_\varepsilon = I. \quad (9)$$

Associate the n^2 -dimensional vector space of matrices to every nonterminal. For a rule

$$X \rightarrow u_0 Y_1 u_1 \cdots Y_k u_k$$

with terminal blocks u_j , its constructor map is

$$F_p(Z_1, \dots, Z_k) = M_{u_k} Z_k M_{u_{k-1}} \cdots Z_1 M_{u_0}. \quad (10)$$

Each child matrix occurs exactly once, so the map is multilinear. Repeated nonterminal names cause no problem: $X \rightarrow YY$ has two independent arguments Z_1, Z_2 , not a quadratic map $Z \mapsto Z^2$. The prototype accepts mixed terminal/nonterminal token sequences directly and uses row-major vectorization. It constructs the tensor by evaluating (10) on every tuple of standard basis matrices. This is simple and auditable. For larger matrices, a production backend should exploit the matrix-product structure directly instead of expanding all tensor coordinates; the closure certificate can retain the same basis-tuple identities.

Proposition 6.1 (Exact CFG reduction). *For a finite CFG with the above terminal semantics, each derivation tree evaluates under the compiled multilinear signature to the matrix of its terminal*

yield. Consequently, zero-observation checking over the start nonterminal is equivalent to checking that observation over every generated terminal word.

Proof. Induct on a derivation tree. For a nullary production the assertion is the definition of the matrix of its terminal block. For a recursive production, substitute the child induction hypotheses into (10); the result is exactly the reversed product for concatenating the terminal blocks and child yields under (9). Every generated word has a finite derivation tree and every derivation tree has a generated yield. $\square$

Ambiguity does not invalidate this reduction. All parses of the same word give the same matrix product. However, the worker is *not* summing the weights of all parses of a word. It is not a decision procedure for arbitrary CFG language equivalence, nor for general weighted-CFG equivalence with aggregate parse semantics. Those are different questions.

6.2 Observations and comparisons

For a fixed initial state v and output row o , the observation oMv is linear in the entries of M , hence is a valid target. To compare two linear or affine machines along every legal trace, use their block-diagonal terminal matrices and subtract their outputs. Affine machines are homogenized first. Several output coordinates become several target rows; several legal entry nonterminals become several target sorts.

A native use case is a pair of implementations whose calls and returns must be properly nested. Another is a collection of mutually recursive syntax constructors whose transfer effects are linear on a state vector. The grammar describes the permitted nesting; matrices describe the effect of a terminal event. The proof is insensitive to execution length but sensitive to the exact original grammar.

6.3 Orientation errors need a noncommutative control

The balanced-translation example alone cannot reliably detect every multiplication-order error, because its matrices commute. The delivered suite therefore includes a grammar whose sole word is ab and matrices

$$A = \begin{pmatrix} 1 & 1 \\ 0 & 1 \end{pmatrix}, \quad B = \begin{pmatrix} 1 & 0 \\ 1 & 1 \end{pmatrix}.$$

Then

$$BA = \begin{pmatrix} 1 & 1 \\ 1 & 2 \end{pmatrix}, \quad AB = \begin{pmatrix} 2 & 1 \\ 1 & 1 \end{pmatrix}.$$

The observation $M_{11} - M_{22}$ is -1 under the correct convention and 1 under the reversed convention. The checker accepts the former and rejects a witness claiming the latter. This is a semantic test of the source adapter, not simply a test that both matrix routines use the same convention.

6.4 Exponential terminal length with a small exact witness

Use scalar terminal matrix $M_a = (1)$ and nonterminals $N_0, \dots, N_{60}$ with rules

$$N_0 \rightarrow a, \quad N_i \rightarrow N_{i-1}N_{i-1} \quad (1 \leq i \leq 60).$$

The unique start word is $a^{2^{60}}$, and its scalar observation is 1. The false zero target therefore has no shorter terminal counterexample at all. The prototype constructs 61 DAG nodes and the checker independently reports

$$\text{word length} = 1,152,921,504,606,846,976,$$

$$\text{expanded derivation nodes} = 2,305,843,009,213,693,951.$$

All rational values remain 1 in this fixture, so it isolates structural compression from coefficient growth. This separation is why the coefficient-explosion test uses terminal value 2 as a different case rather than conflating the two phenomena.

7 The multi-affine recursive-source frontend

7.1 A mechanically checkable source fragment

For each sort, suppose the raw summary has m_s rational coordinates. An operation is given by explicit rational polynomials in its child summaries. The frontend accepts exactly those monomials that use at most one coordinate from each child slot. Thus a term x_1y_2 from different child slots is accepted, whereas x_1x_2 from the same child slot is rejected, even if the coordinate indices differ. Degree is measured *per child slot*, not merely per scalar variable.

Represent a lifted child by $(c_j, x_{j,1}, \dots, x_{j,m_{s_j}})$. Replace a monomial

$$a \prod_{j \in J} x_{j,\alpha_j}$$

by

$$a \left(\prod_{j \in J} x_{j,\alpha_j} \right) \left(\prod_{j \notin J} c_j \right). \quad (11)$$

Set the output's homogeneous coordinate to $\prod_j c_j$. Every term is now linear in each complete child slot. On actual lifted summaries all $c_j = 1$, so the original operation is recovered. With no children the empty product is one and the operation is a constant vector.

Proposition 7.1 (Homogenization correctness). *For every accepted raw operation f_p , its compiled map F_p is multilinear and satisfies*

$$F_p((1, x_1), \dots, (1, x_k)) = (1, f_p(x_1, \dots, x_k)).$$

Hence the compiled fold agrees with the raw fold on every finite derivation.

Proof. Equation (11) has exactly one degree-one factor from every input slot, so scaling or adding a slot distributes as required. Substituting $c_j = 1$ leaves the original monomial. Summing proves the operation equation, and structural induction proves the fold equation. $\square$

The executable frontend is not a general polynomial parser. Its source object explicitly lists rational coefficients and (child slot, coordinate) factors. This keeps the acceptance boundary small. A future Lean frontend can obtain the same object by a proof-producing normalization of constructor equations; that extra translation is not implemented here.

7.2 What is rejected, and why

A map $x \mapsto x^2$ of a single child is not multilinear. Reinterpreting it as a two-child multiplication would admit independent arguments not present in the original recursion. One may route such a goal to Forge’s existing polynomial-ideal lane, or develop a separately certified lift whose finite closure is established. Simply increasing the tensor budget cannot make the incorrect translation exact.

Guarded recursion needs the same care. A source branch selected by an arithmetic test is not an unconditional grammar operation unless the source adapter proves that every grammar derivation is realizable under the guards. A finite sort refinement can encode some control states exactly. Otherwise, a grammar over-approximation can be useful for positive proofs only; a returned derivation must still pass the real guard conditions before becoming a source counterexample.

Natural-number subtraction is another boundary. The raw polynomial expression $\ell - i - 1$ uses rational subtraction, not truncated subtraction in $\mathbb{N}$. A Lean frontend should transport a natural equality such as $L = I + 1$ through an injective cast, not silently translate an arbitrary `Nat.sub` into ring subtraction. Similarly, a recurrence over machine integers requires overflow semantics or a proved no-overflow condition, not unbounded rational arithmetic.

7.3 A practical recognition algorithm for Lean

At a future proof-planning boundary, inspect the equations of the fold or recursively defined summary already selected by the structural worker. Build a dependency graph from recursive result occurrences to output polynomial coordinates. Normalize each constructor equation using proved ring equalities. Annotate each normalized monomial with the child slots it uses. Reject repeated slots, unresolved conditional semantics, unsupported coefficients, and unproved casts. Homogenize accepted monomials and emit a theorem equating the original constructor’s lifted value to its tensor map.

The output is not merely a tensor. It is a tensor plus one compatibility theorem per constructor, an observation-compatibility theorem, and a source-derivation correspondence. This is a concrete compilation task with a decidable syntactic acceptance test; it does not ask an external solver to decide whether an arbitrary recursive Lean program is multilinear.

8 Exact observable quotients by one-hole contexts

Implementation status. Implemented in `minimize`; quotient preservation is checked independently. The checker certifies preservation, not minimality.

8.1 Why forward reachability is only the first compression

A reachable summary may contain information that never influences any requested observation. Forward closure removes unreachable linear directions, but it does not remove invisible reachable directions. For a unary transition system, backward observation closure finds them. With a binary constructor, the influence of one child depends on the other child’s value, so the backward step must quantify over reachable siblings.

A *one-hole context* of input sort s is a well-sorted derivation with exactly one child subtree replaced

by a hole of sort s , ending at a sort with a listed observation. Filling the other positions with fixed derivations and composing with the observation gives a linear functional on V_s . Two reachable summary states are contextually indistinguishable when every such functional gives the same result.

8.2 Work in reachable coordinates

First run the forward closure without stopping on nonzero observations. Let its independent reachable columns be B_s and write r_s for their number. The forward certificate's coordinates define a multilinear operation

$$H_p : \mathbb{Q}^{r_{s_1}} \times \cdots \times \mathbb{Q}^{r_{s_k}} \rightarrow \mathbb{Q}^{r_s},$$

whose value on a standard-basis tuple is $h_{p,i}$. Thus

$$F_p(B_{s_1}x_1, \dots, B_{s_k}x_k) = B_s H_p(x_1, \dots, x_k). \quad (12)$$

The initial context rows at sort s are the requested rows qB_s .

For a row c at an operation's output sort, choose an input position j and standard-basis values for every other input. Pull c back to a row at input sort s_j :

$$\rho(x) = c H_p(e_{i_1}, \dots, e_{i_{j-1}}, x, e_{i_{j+1}}, \dots, e_{i_k}). \quad (13)$$

Admit this row when it is independent of rows already stored at s_j . Repeat until no new row is admitted. Each sort admits at most r_s rows, so the process terminates. Unlike an ordinary backward matrix multiplication, every pullback is indexed by the chosen hole and the sibling-basis tuple.

```

B, H := completed forward reachable-span closure
for each target (s,q): admit_context_row(s, q * B[s])
while a new context row (out,c) is pending:
  for each operation p with result sort out:
    for each input position j:
      for each tuple of sibling basis indices:
        rho[k] := c * H[p](siblings, e_k in position j)
        admit rho at the sort of position j if independent

```

Let $\mathcal{C}_s$ be the final context-row space and let C_s be a full-row-rank matrix of its stored generators. Because the forward columns denote actual derivations, every newly generated row comes from an actual one-hole context. Conversely, multilinearity in the siblings expands any reachable sibling value into those columns; induction on contexts then shows that every actual context row lies in $\mathcal{C}_s$.

8.3 The context kernel is a congruence

Define $K_s = \ker C_s$ on reachable coordinates.

Theorem 8.1 (Well-defined observable quotient). *The family (K_s) is compatible with every H_p : changing an input by an element of its K_{s_j} changes the output by an element of K_s . Consequently, the operations descend to quotient spaces $\mathbb{Q}^{r_s}/K_s$, and all requested observations descend with them.*

Proof. First change only the j th input by $z \in K_{s_j}$. By linearity in that slot, the output difference is $H_p(x_1, \dots, z, \dots, x_k)$. Pair it with any $c \in \mathcal{C}_s$. Expand the other inputs in standard bases. Each

resulting functional of z is a linear combination of the pullbacks (13), all in $\mathcal{C}_{s_j}$, so it vanishes on z . Therefore the output difference lies in K_s .

For several changed inputs, replace them one at a time and sum the differences. This telescoping argument avoids incorrectly discarding multilinear cross terms. Target rows belong to the initial context spaces, so they annihilate K_s and descend to the quotients. $\square$

There is an exact minimality interpretation, with a carefully restricted scope. Any linear summary quotient of the *reachable* spaces that preserves constructor composition and all observations must distinguish every pair distinguishable by a one-hole context. Its kernel is therefore contained in K_s , so its dimension is at least $\text{rank } C_s$. The constructed quotient attains that dimension. This is not a claim about arbitrary nonlinear encodings, changed source languages, or arbitrary proof search states. Moreover, the delivered checker does not verify that the producer found the least reachable space or the full context space; it verifies the weaker and sufficient preservation diagrams below.

8.4 Constructing a concrete small model

Let $k_s = \text{rank } C_s$ and choose a section $R_s : \mathbb{Q}^{k_s} \rightarrow \mathbb{Q}^{r_s}$ with

$$C_s R_s = I_{k_s}. \quad (14)$$

Use coordinates $z = C_s x$ for the quotient and define

$$\overline{H}_p(z_1, \dots, z_k) = C_s H_p(R_{s_1} z_1, \dots, R_{s_k} z_k). \quad (15)$$

Since $x - R_s C_s x \in K_s$, Theorem 8.1 implies

$$C_s H_p(x_1, \dots, x_k) = \overline{H}_p(C_{s_1} x_1, \dots, C_{s_k} x_k). \quad (16)$$

For each original target, put $\beta = (qB_s)R_s$. The target diagram is

$$qB_s = \beta C_s. \quad (17)$$

The prototype constructs the sections by exact elimination and materializes the resulting quotient tensors. Zero-dimensional observable quotients are allowed: if every listed observation is identically zero on every context, the whole reachable space can disappear.

8.5 What the quotient certificate actually checks

A quotient certificate contains a forward enclosure, matrices C_s and R_s , a proposed smaller multilinear model, its output rows β , and matrices L_s satisfying

$$L_s B_s = I_{r_s}. \quad (18)$$

The last identity is deliberate. A plain cover can have dependent generators, but interpreting C_s as a function of a state in the span requires unique reachable coordinates, or an additional proof that C_s kills every representation dependency. The delivered format takes the simpler route: a finite left-inverse identity certifies column independence without asking the checker to perform a rank calculation.

The checker verifies the enclosure, (18), (14), every basis-tuple instance of (16), and (17). It verifies that the original and quotient signatures align operation by operation and target by target. Multilinearity and induction then imply equality of all requested observations on every original derivation. It does not run backward context search, compute ranks, or certify that no smaller model exists.

8.6 A recorded compression with visible semantics

The noisy-tree fixture starts with raw coordinates (L, I, U, V) . Leaves have $(1, 0, a, 0)$ for labels $a \in \{0, 1, 2\}$. A node adds its children's L values, adds their I values and one, adds their U values, and sets

$$V' = V_1 + V_2 + U_1 U_2.$$

The target observation is L . After homogenization the ambient dimension is five. The forward closure finds dimension four: the old relation $L - I - c = 0$ removes one direction. The backward context closure leaves exactly two observable coordinates. The independent quotient checker accepts the resulting model through 19 constructor identities and the quotient diagrams.

In familiar coordinates a two-dimensional model is simply (c, L) with leaf $(1, 1)$ and node $(c_1 c_2, L_1 c_2 + c_1 L_2)$. The algorithm need not choose this familiar basis; any invertible coordinate change within the observable quotient is acceptable. The point is exact elimination of U, V and the dependent count coordinate, not a floating-point low-rank approximation.

9 A concrete Lean integration plan

Implementation status. Design only, except for an included hand-written replay specimen. No Lean or Lake executable was available; that specimen has not been elaborated.

9.1 The missing theorem is finite span closure

The first useful Lean milestone is not a monolithic tactic. It is a theorem that turns a finite table of constructor images into closure of the spans of the supplied generators. The mathematical proof is Lemma 3.1. Mathlib already provides `MultilinearMap`, including linearity in individual slots and a `map_sum` theorem for distributing finite sums in every input [8]. The implementation should specialize those results to finite coordinate vectors and finite generator indices.

A suitable theorem interface, written here as a mathematical specification rather than supposedly compiled Lean syntax, is:

Given a multilinear F and finite generator maps B_j , if for every generator-index tuple $\mathbf{i}$ there are supplied coefficients $h_{\mathbf{i}}$ with $F(B_1 i_1, \dots, B_k i_k) = \sum_a (h_{\mathbf{i}})_a B_0 a$, then F maps the product of the input spans into the output span.

For binary trees, instantiate the existing `ForgeClosure.tree_property` with membership in that span and the target kernel. For other inductive datatypes, use their existing recursors with the same finite span-closure theorem. Do not introduce another generic cover theorem merely to rename the induction already available in Forge [4].

The finite index product must preserve arity-zero behavior: the type of choices for no input positions has one inhabitant. A version that expands into a sum indexed by an accidentally empty container would erase the nullary proof obligation. This should be a dedicated Lean regression test, not left to informal reasoning about empty lists.

9.2 Proposed module boundaries and acceptance gates

New module	Concrete responsibility and completion criterion
<code>Forge/Multilinear/Signature.lean</code>	Finite sorts, coordinate dimensions, typed operation arities, tensor interpretation, and finite derivations. Prove evaluation agrees with each explicitly supplied source algebra.
<code>Forge/Multilinear/Certificate.lean</code>	Bounded data for generators, constructor coordinates, target rows, and derivation DAGs. All indices are checked against original signatures.
<code>Forge/Multilinear/Soundness.lean</code>	Prove the span-extension theorem and combine it with source recursors. Prove the finite quotient-diagram theorem separately.
<code>Forge/Multilinear/Check.lean</code>	Exact rational identity checker and a soundness theorem. Tests must reject incomplete tuple tables, altered targets, and malformed DAG edges.
<code>Forge/Frontends/MultiAffine.lean</code>	Recognize degree at most one per recursive child slot; emit homogenization and observation-transport proofs for the original constructor equations.
<code>Forge/Frontends/ContextFree.lean</code>	Compile original grammar productions to matrix operations and prove the derivation/yield correspondence. Test a noncommutative orientation fixture.
<code>Forge/Workers/Grammar.lean</code>	Submit an independently bound source problem to the untrusted producer, replay the result, and close only the original goal through the correspondence theorems.

This plan is deliberately a worker integration into the existing controller, not a fresh scheduler, global search language, or authority model. Forge already has substantial designs for those concerns; the new acceptance gates concern the algebra and the source translation.

9.3 A staged proof-construction strategy

A first implementation can avoid a generic reflected checker by emitting ordinary Lean terms that prove the finite identities. Each rational equality is closed through ordinary proof-producing normalization; each constructor table becomes a finite case split over explicit indices. The span-closure theorem then assembles the universal proof. The user should see a normal proof of the original statement, not an external verdict.

The package’s `lean/ReplayLeafCount.lean` illustrates this route. It imports Mathlib and Forge’s existing cover theorem, represents the discovered two-column span by

$$\text{rep}(a, b) = (a + b, a + 2b, b),$$

and proves the collected bilinear constructor identity

$$\begin{aligned} F(\text{rep}(a, b), \text{rep}(c, d)) &= \text{rep}(\alpha, \beta), \\ \alpha &= -ad - bc - 2bd, \\ \beta &= ac + 2ad + 2bc + 3bd. \end{aligned}$$

These coefficients are exactly the bilinear extension of the recorded table (2). The specimen uses the existing `tree_property` to conclude the zero observation for its explicitly supplied tree algebra.

It does not claim to be generated by a Lean reifier or to prove a separate natural-number count function’s specification automatically. It is hand-written, contains no admitted proof holes, and remains **uncompiled**; textual absence of proof holes is not a substitute for elaboration or an axiom audit.

Once this finite proof path is working, a reflected checker can reduce proof-term size and repeated elaboration. The reflected theorem should quantify over the original signature and semantics, take a Boolean acceptance equality for the certificate, and imply the universal source observation after the separately proved correspondence. Search need not be formalized. The retained Python producer may be replaced by FLINT-backed or modular search without changing the logical acceptance theorem.

9.4 The source-correspondence chain

For a real Lean goal, four separate facts must line up:

$$\begin{array}{lcl} \text{original expression} & \xrightarrow{\text{proved constructor laws}} & \text{source algebra,} \\ \text{source algebra} & \xrightarrow{\text{proved compilation}} & \text{multilinear semantics,} \\ \text{multilinear semantics} & \xrightarrow{\text{checked certificate}} & \text{observation theorem.} \end{array}$$

The CFG adapter additionally needs a proof that original permitted traces correspond to the chosen derivations. Mathlib’s current CFG module supplies `ContextFreeRule`, `ContextFreeGrammar`, `Derives`, and `language`; a bridge from the new finite derivation representation to that rewriting-based interface must be proved, not assumed from matching names [9].

A proof of $q[t] = 0$ over rational coordinates does not identify what the original Lean program computed. The constructor laws do that. Nor does a proof over every parse tree by itself establish a theorem about an unrelated parser’s accepted language. The derivation/yield correspondence does that. Keeping these obligations separate makes the prototype’s present limits precise and gives implementers concrete tasks rather than a vague instruction to “reify correctly.”

9.5 Library choices

The executed package uses only Python’s standard library: `fractions.Fraction`, finite products, queues, JSON, and the unittest framework. Exact rational arithmetic is sufficient for the small experimental dimensions. No NumPy, SciPy, SymPy, SMT, or numerical-rank oracle is needed to run search or replay.

For a production producer, FLINT’s `fmpq_mat_t` is a suitable rational matrix substrate. Its documented routines work with canonical rational entries and use denominator clearing internally for heavy integer computations [10]. That is a proposed backend, not an installed or benchmarked dependency of this package. Sparse tensor contraction and operation-structured matrix products should remain outside a dense matrix abstraction when expansion would destroy useful structure.

For Lean, use Mathlib’s finite-dimensional vector, matrix, submodule, and multilinear infrastructure. The essential named multilinear APIs were checked in current generated documentation; they have not been compiled against the repository’s pinned Mathlib revision. The first integration branch must pin its dependencies and resolve any API differences by compilation. `ring` and rational normalization are candidate replay tools, not a reason to trust an external normal form without a proof.

A larger producer may use modular arithmetic to suggest bases or rational-reconstruction candidates. The final identities must still be checked exactly. A modular nonzero value of a concrete, denominator-safe witness can sometimes guide exact witness replay, but a collection of modular zero tests is not a universal rational proof. The current implementation has no modular lane.

10 Leant and Djex: a narrow, useful connection

The reviewed Forge comparison already incorporates lessons about verified candidate ownership, solver observations, lexical identity, re-elaboration, and authority separation. Repeating them as new inventions would violate the purpose of this extension [5]. The opportunity here is narrower: provide a *complete semantic comparison procedure on an explicit recursive algebra fragment* to a synthesizer that already knows how to propose typed candidates.

The inspected September 14 diagnostics in Leant and Djex distinguish an original indexing query from easier component queries. Function-valued folds and the required integer-case step were available separately, while the original bounded composition search still had no accepted solution. Their canonical repository revisions are recorded separately from Leant’s pinned native Djex dependency [11, 12]. Nothing in the present prototype closes that indexing query or the simultaneous-universe query.

A concrete integration is possible when a candidate and specification each come with proved folds into accepted multi-affine summaries. Pair the summaries, compare their observations by the new worker, and return either a full finite certificate or a concrete derivation rejecting the candidate. A counterexample DAG can be added to a candidate-filtering corpus without expanding it. An accepted quotient can remove irrelevant state components from subsequent *supported* comparisons. These operations add semantic information that mere type inhabitation does not supply.

However, a Church-encoded value is not automatically interchangeable with an inductive tree or list. The original encoding may have different universe behavior or may require a representation/-parametricity law unavailable in the current context. That bridge remains a required hypothesis or theorem. Similarly, a tiny abstract quotient does not justify merging two Lean candidates with distinct dependent behavior outside the declared observations. The observation set and source correspondence are part of the claim’s scope.

A useful first experiment in Leant would therefore be a new, explicitly named algebraic-fold benchmark, not relabeling the unresolved original indexing case as solved. Keep its candidate generation budget fixed, compare candidate rejection by concrete small samples with rejection by compressed certified derivations, and report the source-fragment coverage separately from the number of generated terms. This experiment is proposed, not performed here.

11 Executed experiments and what they establish

11.1 Environment and reproducibility

All recorded algorithms and tests ran under Python 3.13.5 using standard-library-only execution. The suite is invoked with `python -S`, disabling site-package initialization. The independent replay experiment additionally uses Python’s isolated mode and copies *only checker.py* to a fresh temporary directory. Search and compiler modules are absent there. The exact environment is recorded in `results/environment.json`.

Neither `lean` nor `lake` was present. No certificate in this package has been checked by Lean’s kernel, and no native tactic comparison was performed. The prior repository’s own Lean elaboration results remain prior results; they are not reclassified as tests of this extension.

11.2 Recorded fixtures

The following table is generated from `results/fixtures.json`. Byte counts are for the certificate serialized by `json.dumps(..., sort_keys=True)` with its default separators, not the size of the pretty-printed file or the ZIP member. They exclude the independently supplied original problem.

Fixture	Certificate bytes	Identities / DAG nodes	Result
Tree-count identity	384	5	PROVED
Balanced traces	247	3	PROVED
Changed return matrix	176	2	REFUTED
Noncommuting order control	149	1	REFUTED
Exponential-length witness	2,160	61	REFUTED
Noisy-tree quotient	2,451	19	preserved
Zero-observation quotient	778	5	preserved
Language restriction removed	176	2	REFUTED

Table 3: Recorded fixture certificates. Positive and quotient rows count constructor identities; negative rows count stored witness DAG nodes.

For the two positive fixtures, the constructor-identity counts are five for the tree-count theorem and three for the balanced-trace theorem. Source/tensor audits perform ten and 21 ambient basis-tuple comparisons respectively. These audit counts and constructor-closure counts are different quantities and should not be added as if they measured solved goals.

The noisy quotient has ambient dimension five, reachable dimension four, and observable dimension two. Its forward enclosure has 19 constructor identities; context search considers 16 sibling-conditioned pullbacks. The all-zero-observation quotient has dimension zero, which is a legitimate model, not a missing-data sentinel.

11.3 An exhaustive differential oracle

The differential suite generates 240 acyclic, three-sort signatures from seeds 10000 through 10239. Each signature has dimension two or three at every sort, two nullary alternatives at the first sort, and unary/binary operations referring only to the immediately preceding sort thereafter. Coefficients are small exact rationals. The finite derivation languages are exhaustively evaluated by a separate evaluator using no span algorithm and neither the producer’s nor checker’s evaluation helper.

Half the generated cases deliberately force the final output coordinate to zero in every operation and use a nonzero target selecting it. The other half use unrestricted random coordinates; all 120 of those recorded instances have a counterexample. The results are therefore 120 PROVED and 120 REFUTED, with all 240 certificates accepted and all 240 classifications matching exhaustive evaluation. These are manufactured correctness tests, not a representative Lean theorem corpus or an estimate of an application’s success probability.

Enumeration deduplicates identical summary values. This is sound for the exhaustive oracle because all parent operations depend only on those summaries. It does not discard a different value merely

because it happens to have the same current target observation. The 240 cases collectively enumerate 9,211 distinct per-sort values; that count is supplementary workload information, not an additional number of tests.

11.4 A separate paired-derivation quotient oracle

Forty further generated acyclic problems, seeds 20000 through 20039, are minimized. Every proposed quotient passes the independent diagram checker. A second finite-language calculation evaluates the original and quotient models on *matching derivations*, retaining pairs of states at every sort. It then compares the corresponding requested observations. All 40 cases agree, covering 1,180 distinct observed state pairs.

Comparing just the *sets* of possible outputs would be insufficient: a faulty quotient could permute which input receives which output while leaving the output set unchanged. Pairing each recursive derivation avoids that weakness. The oracle is still only finite test evidence for the implementation; the universal justification of a checked diagram is the theorem in Section 8.

11.5 Negative controls and parser tests

The twenty designed invalid objects cover missing and duplicate closure entries, false coefficients, an altered target, noncanonical rationals, floats, Boolean indices, unknown fields, a source/tensor mismatch, missing generators, cyclic witness references, a false claimed observation, a bad root, missing children, a transplanted witness from another transition system, the reversed multiplication convention, bad left inverses, bad quotient sections, and changed quotient observation or constructor data. All twenty are rejected. Duplicate JSON keys are tested separately through the actual loader.

These are deliberately invalid mutations, not a claim that every random perturbation must be rejected. Redundant generators, a different valid basis, or another valid proof of the same theorem should often be accepted. The test suite includes redundant-cover acceptance precisely to keep “aggressive validation” from becoming an accidental restriction of the mathematical certificate language.

Three resource tests return UNKNOWN: zero candidate budget, zero basis budget, and exact coefficient explosion under a 64-bit cap. Additional tests cover no finite derivations, a zero nullary, repeated input sorts, ternary constructors, same-child nonlinear-source rejection, zero-dimensional quotients, and exact echelon-coordinate reconstruction. All 18 unittest methods pass, with no failures or errors; several methods contain many parameterized subcases. The 18 methods, 240 differential cases, 40 quotient cases, and 20 invalid objects are overlapping views of the suite and must not be pooled into a single success count.

11.6 Isolation and the strength of the evidence

Eight stored fixture certificates were replayed in fresh isolated Python processes containing only the checker. All were accepted. This shows that replay does not need the producer, compiler, their caches, or third-party packages. It does not show that Python, `Fraction`, the operating system, or the checker implementation is formally verified. The checker and producer also share a mathematical representation and the standard rational arithmetic library, even though their executable functions are separate.

The archive records individual timings for inspection but makes no speedup claim. There was no randomized timing protocol, no alternative production backend, and no run of `grind`, `Aesop`, or any other Lean tactic. The supported conclusion is *algorithmic feasibility and reproducible exact evidence on the delivered fragments*, not measured superiority over an existing prover.

12 Integration priorities, limits, and further development

12.1 A minimal merge that would be worth shipping

The first merge should add the typed signature and positive certificate theorem, integrate the binary-tree source algebra using the existing cover theorem, and compile the exact tree-count certificate. Completion means an ordinary Lean theorem over the original source algebra, a recorded axiom inventory under the project’s normal policy, and regression rejection of a changed closure coefficient. Merely compiling data structures or a theorem about an unrelated abstract model does not close that milestone.

The second merge should add multi-affine source recognition and constructor-compatibility proofs, followed by the context-free matrix adapter. Completion requires a true balanced-trace theorem, a checked legal counterexample for the changed return, and the noncommutative orientation test. The original language restriction must remain visible in the theorem statement.

The third merge should add exact quotient diagrams. Completion requires replay of both the noisy two-dimensional quotient and the zero-dimensional quotient, plus proof that the original observations agree on every derivation. A check that only verifies the smaller model’s internal consistency is insufficient.

Only after these steps should the worker be scheduled automatically by Forge. A good trigger is a recognized recursive source algebra with a nonzero degree at most one per child slot, or a supplied CFG trace restriction with matrix actions. Inputs outside that fragment should route to existing workers rather than be force-fit into an unsound encoding.

12.2 Where the practical limits arise

High arity makes the tuple product (6) large. Large matrix-valued grammars make even the ambient tensor audit expensive. Exact coefficients can grow far faster than the generator count. These are distinct resource dimensions and should have separately reported cutoffs. A single vague fuel number conceals whether the obstacle is a missing algebraic representation, an exponential tuple table, or integer growth.

Sparse tensors, structured matrix operations, and per-sort dependency slicing can reduce actual work without changing the acceptance language. Quotient construction can reduce repeated downstream work, but it first pays for forward reachability and backward contexts; running it for every trivial goal would be poor scheduling. The prototype has not measured an amortization break-even point.

A finite projection of an unbounded index, an approximate floating-point rank, or a bounded unrolling of recursion is not an equivalent substitute for the exact fragment. Such tools may propose candidates or counterexamples, but their conclusions require their own soundness arguments. The present procedure decides equations of linear observations, not arbitrary inequalities, analytic limits, nonpolynomial semantics, or dependent-type inhabitation.

12.3 Two next algorithmic improvements with explicit contracts

Structure-preserving CFG contraction. Keep a production as the matrix-product expression (10). During search and replay, contract reachable basis matrices directly. Avoid the n^{2k} ambient standard-basis expansion when a Lean theorem has already established multilinearity and source equivalence of that structured operation. Acceptance still checks every reachable generator tuple. This changes the implementation cost, not the theorem; the present Python frontend audit deliberately pays the larger simple cost because it has no Lean source theorem.

Certified quotient reuse. When several goals share a source grammar and constructor semantics but differ in target rows, retain a proved enclosing basis and its constructor coordinates. Reuse it only as an enclosure certificate, then rerun target checks and the context closure for the new observation set. Adding observations can increase observable dimension, so an old smaller quotient may no longer suffice. Its diagrams remain mathematical facts, but they do not establish preservation of a new output until that output's target diagram is checked.

Neither proposal authorizes accepting a cached search verdict. The reusable objects are explicit finite identities whose relationship to the current original source must still be established by the existing Forge boundary.

13 Reproducing and extending the package

The delivered code targets Python 3.11 or newer and uses no third-party packages. Python 3.13.5 is the version actually tested. From the archive root:

```
cd prototype
python -S -m unittest -v tests
python -S run_experiments.py
```

The second command writes a new `reproduced-results/` directory and does not overwrite the archived `results/` by default. Its JSON summary distinguishes fixture certificates, differential cases, quotient cases, invalid objects, and quota outcomes.

To replay a stored certificate without search:

```
python -I -S checker.py \
  ../results/fixtures/balanced_dyck.problem.json \
  ../results/fixtures/balanced_dyck.certificate.json
```

To solve another problem in the explicit schema and save its certificate:

```
python -S solve.py input.problem.json --certificate output.certificate.json
python -S solve.py input.problem.json --quotient \
  --certificate output.quotient.json
```

The command-line wrapper decodes and audits the original source, runs search, and replays the resulting certificate before reporting success. It refuses to overwrite an existing certificate path. Exit code zero denotes a replayed abstract result, code three denotes a cutoff, and code two denotes malformed input, replay rejection, or an input/output error. An abstract refutation is a successfully answered query, not a process failure.

The article uses ordinary pdfLaTeX and includes its small generated results table:

```
cd article
pdflatex -interaction=nonstopmode -halt-on-error forge_grammar_closure.tex
pdflatex -interaction=nonstopmode -halt-on-error forge_grammar_closure.tex
```

The Lean specimen has different prerequisites: a compatible Mathlib project and the inspected Forge cover module on its import path. It must be compiled before anyone treats it as evidence. `docs/LEAN-STATUS.json` records that it has not been compiled here.

14 Conclusion

The useful next step for Forge is not another broad promise to combine solvers. It is an exact new worker with a clear boundary: independently recursive children, finite rational multilinear summaries, and linear observations. Reachable-span closure converts an unbounded structural problem into finite identities. Matrix-valued grammar compilation preserves context-free input restrictions. One-hole-context closure then removes information invisible to every supported surrounding computation.

The delivered prototype implements all three computations, keeps counterexamples as original derivation DAGs, independently audits its two source languages, and replays certificates without importing search. Its tests exhibit both nontrivial positive certificates and exact failures that bounded word enumeration would mishandle. The remaining Lean work is sharply identified: the finite span theorem, reflected or proof-term identity replay, and source-correspondence adapters. Those are tractable implementation targets, not accomplished results. Their completion would give Forge a genuine additional proof capability while preserving the repository's existing trust discipline.

A Certificate schema and implementation map

The explicit schema uses strings for sort names and canonical rational coefficients. Tensor entries have an output-coordinate index, one input-coordinate index per child occurrence, and a coefficient. Operation indices always refer to the original problem's operation array.

```
{
  "schema": "forge.mlc.problem.v1",
  "name": "example",
  "sorts": {"T": 3},
  "operations": [
    {"name": "leaf", "out": "T", "inputs": [],
     "terms": [{"out": 0, "in": [], "q": "1"}, ...]},
    {"name": "node", "out": "T", "inputs": ["T", "T"],
     "terms": [{"out": 0, "in": [0, 0], "q": "1"}, ...]}
  ],
  "targets": [{"sort": "T", "q": ["-1", "1", "-1"]}
}
```

Ellipses above are explanatory, not valid JSON. Fully executable examples are in `results/fixtures/`. The array storage of a basis is a list of *columns*, each serialized as its coordinate vector; equations in the article use the corresponding column matrix.

```
{
  "schema": "forge.mlc.certificate.v1",
  "kind": "invariant",
  "basis": {"T": [["1", "1", "0"], ["1", "2", "1"]]},
  "closures": [
    {"op": 0, "children": [], "coeff": ["1", "0"]},
    {"op": 1, "children": [0, 0], "coeff": ["0", "1"]},
    {"op": 1, "children": [0, 1], "coeff": ["-1", "2"]},
    {"op": 1, "children": [1, 0], "coeff": ["-1", "2"]},
    {"op": 1, "children": [1, 1], "coeff": ["-2", "3"]}
  ]
}
```

A certificate of kind `enclosure` has the same fields but makes no target claim. It is used as an internal reusable object by quotient construction. A `counterexample` has `nodes`, `root`, `target`, and `claimed_value`. A `quotient` contains an `enclosure`, `maps`, `sections`, `left_inverses`, and a complete smaller `model`. These different claims are not interchangeable.

File	Role
<code>search.py</code>	Incremental exact span, semi-naive constructor scheduling, reachable witnesses, context closure, quotient synthesis.
<code>compilers.py</code>	Explicit multi-affine homogenization, CFG matrix compiler, and named fixtures.
<code>checker.py</code>	Strict decoding, independent source semantics, finite coverage and identity checking, DAG replay, quotient diagrams; no producer imports.
<code>tests.py</code>	Unit tests, exhaustive finite-language oracle, paired-derivation quotient oracle, and invalid-object construction.
<code>run_experiments.py</code>	Recorded fixtures, all generated cases and certificates, mutation outcomes, isolated subprocess replay, and environment manifest.
<code>solve.py</code>	Command-line input audit, untrusted search, mandatory replay, and certificate output.

B Snapshot and evidence notes

The repository snapshots inspected for this article are:

Forge

c98e47c5f804e92880fc1d0e378c1b95832b685c.

Leant

6bf05ad78c467989e68290f2d08bbbed40802d485.

Djex canonical repository

e8778f4ebd63e1f9b9b410fa4de8d14a8a04c9e5.

The reviewed Leant diagnostic identifies its native Djex dependency as `e237e866`, which is not the same object as the canonical Djex repository snapshot above. The review keeps these identities

separate. The proposed Lean integration uses the Forge baseline’s reported Lean/Mathlib target, but this package has not built those dependencies.

Some web interfaces returned truncated source previews, and a screenshot request for the primary paper failed. The mathematical discussion used the paper’s accessible text and abstract, not an unseen figure. The directory `docs/` records the source URLs, audit scope, and implementation status. No third-party source archive or font file is redistributed.

References

- [1] Vladimir Reshetnikov. *Forge: README*, inspected commit `c98e47c5`. September 2026. <https://github.com/VladimirReshetnikov/Forge/blob/c98e47c5f804e92880fc1d0e378c1b95832b685c/README.md>.
- [2] Vladimir Reshetnikov. *Forge implementation status*, inspected commit `c98e47c5`. <https://github.com/VladimirReshetnikov/Forge/blob/c98e47c5f804e92880fc1d0e378c1b95832b685c/docs/STATUS.md>.
- [3] Forge contributors. *Finite certificates for unbounded behaviour*, merged article, Section 6, inspected commit `c98e47c5`. <https://github.com/VladimirReshetnikov/Forge/blob/c98e47c5f804e92880fc1d0e378c1b95832b685c/article/sections/06-closure.tex>.
- [4] Forge contributors. *Finite-algebra covers for inductive data*, `Covers.lean`, inspected commit `c98e47c5`. <https://github.com/VladimirReshetnikov/Forge/blob/c98e47c5f804e92880fc1d0e378c1b95832b685c/lean/Forge/Closure/Covers.lean>.
- [5] Forge contributors. *Ideas from Leant and Djex*, inspected commit `c98e47c5`. <https://github.com/VladimirReshetnikov/Forge/blob/c98e47c5f804e92880fc1d0e378c1b95832b685c/docs/IDEAS-FROM-LEANT-DJEX.md>.
- [6] Ines Marušić and James Worrell. *Complexity of Equivalence and Learning for Multiplicity Tree Automata*. Journal of Machine Learning Research **16**(76), 2465–2500, 2015. <https://www.jmlr.org/papers/volume16/marusic15a/marusic15a.pdf>.
- [7] Lean Language Reference. *The grind tactic*. Current documentation inspected September 15, 2026. <https://lean-lang.org/doc/reference/latest/The--grind--tactic/>.
- [8] Mathlib contributors. *Mathlib.LinearAlgebra.Multilinear.Basic*. Generated documentation inspected September 15, 2026. https://leanprover-community.github.io/mathlib4_docs/Mathlib/LinearAlgebra/Multilinear/Basic.html.
- [9] Mathlib contributors. *Mathlib.Computability.ContextFreeGrammar*. Generated documentation inspected September 15, 2026. https://leanprover-community.github.io/mathlib4_docs/Mathlib/Computability/ContextFreeGrammar.html.
- [10] FLINT contributors. *fmpq_mat.h: matrices over the rational numbers*. Development documentation, inspected September 15, 2026. https://flintlib.org/doc/fmpq_mat.html.
- [11] Vladimir Reshetnikov. *Indexing composition after native kind adoption*, Leant, commit `6bf05ad7`, September 14, 2026. <https://github.com/VladimirReshetnikov/Leant/blob/6bf05ad78c467989e68290f2d08bbcd40802d485/docs/reports/2026-09-14-indexing-composition-diagnostics.md>.
- [12] Vladimir Reshetnikov. *Indexing composition after native kind adoption*, Djex, commit `e8778f4e`, September 14, 2026. <https://github.com/VladimirReshetnikov/Djex/blob/e8778f4ebd63e1f9b9b410fa4de8d14a8a04c9e5/docs/reports/2026-09-14-indexing-composition-diagnostics.md>.